Recommendation System

Created	March 17, 2026 12:43 AM
Tags	Recommendation System

Overview of Recommendation System

1. **Amazon 2000:**Collaborative Filtering
2. **Netflix 2007:**Matrix-based approach
3. **Facebook 2011:**Tree-based approach
4. **Google 2016:**Wide & Deep Model (DNN)
5. **TikTok, Taobao, etc.:** Systems based on Wide & Deep model

User	R1	R2	R3	R4	R5
UserA	1	1	1	1	-
UserB	1	-	1	-	-
UserC	-	1	-	-	1
UserD	-	-	1	1	1

- `1` 表示用户对该 restaurant 有兴趣或评分
- 表示缺失值 (unknown / not rated)
- 推荐系统是用来干什么的？

  把“用户-产品”的打分矩阵中的缺失值补齐。
- 补充缺失值的方法有很多，如果是通过已有的评分值来补齐缺失值，那么就是Collaborative Filtering的方法。除此之外，还有很多其他的方法，i.e. content-based methods.

User-based Method

User–Restaurant Matrix

	R1	R2	R3	R4	R5
UserA	1	1	1	1	-
UserB	1	-	1	-	-
UserC	-	1	-	-	1
UserD	-	-	1	1	1

已知Users对某些Restaurants的评价，判断用户对其他Restaurants的兴趣。

评价的分数可以是任意的实数。

1.1 基于与“与你相似的人，向你推荐其他人喜欢的物品”

Step 1：计算用户之间相似度（Cosine Similarity）

Suppose we have user U and V，

$N(u)$denotes the set of items that user U likes：

$$a_{uv} = \frac{|N(u) \cap N(v)|}{\sqrt{|N(u)| \cdot |N(v)|}}$$

Example

- User U: { R1, R2, R3, R4 }
- User V: { R1, R3 }

$$a_{uv} = \frac{2}{\sqrt{4 \times 2}} = 0.7$$

说明：

- $N(u)$ ：用户喜欢的物品集合

User–User Similarity Matrix :

	User A	User B	User C	User D
User A	1	$2/\sqrt{4 \times 2} = 0.7$	$1/\sqrt{4 \times 2} = 0.35$	$2/\sqrt{4 \times 3} = 0.58$
User B	0.7	1	0	$1/\sqrt{2 \times 3} = 0.8$
User C	0.35	0	1	$1/\sqrt{2 \times 3} = 0.8$
User D	0.58	0.8	0.8	1

Step 2：基于相似用户做预测

用前k 个相似的用户中，

对该商品有评价的用户 → 对商品的评价做加权和，作为推荐分数。

Suppose we want to know user U's interest on item x ：

$$p(u, x) = \sum_{v \in S(u,k) \cap M(x)} a_{uv} \cdot r_{vx}$$

符号解释

- $S(u, k)$ ：与用户 U 最接近的 K 个用户
- $M(x)$ ：喜欢物品 X 的用户集合
- a_{uv}：用户相似度
- r_{vx} ：用户 V 对物品 x 的评分

Example (K = 2)

$$p(userA, R5) = 0.7 \times 1 = 0.7$$

$$p(userB, R4) = 0.7 \times 1 + 0.8 \times 1 = 1.5$$

带归一化版本:

$$p(userB, R4) = \frac{0.7 \times 1 + 0.8 \times 1}{1 + 1} \qquad = \frac{1.5}{2} \qquad = 0.75$$

1.2 具体实现方法 - K Nearest Neighbors

模型介绍

- K 表示最接近自己的 K 个数据样本
- KNN 模型可以简单地理解为：由离自己最近的 K 个点来投票决定待分类数据归为哪一类，它是一种基于统计的方法

- 算法的变化性来源于“距离”的定义
  - City-Block Distance, Euclidean Distance, Gaussian Distance
  - 因为求平方根函数是一个单调函数，它对不同距离的绝对值求平方根虽然改变了数值大小，但依然保持了不同距离大小的顺序。
  - 在面对两个向量之间的差异时，L2 比 L1 更加不能容忍这些差异。也就是说，相对于 1 个巨大的差异，L2 距离更倾向于接受多个中等程度的差异。

- 当 k = 1 的时候，k-Nearest Neighbor 分类器就是 Nearest Neighbor 分类器。

从直观感受上来说，更高的 k 值可以让分类的效果更平滑，使得分类器对于异常值有更有抵抗力。

以图像分类器来理解 KNN 的特点

- Nearest Neighbor 算法会拿着测试图片和训练集中每一张图片去比较，然后将它认为最相似的那个训练集图片的标签赋给这张测试图片
- 最简单的方法是逐个像素比较，最后将差异值全部加起来。即计算他们的 L1 距离

- KNN 算法的 predict 过程没什么事情可干，因为它没有 model 可以 train，大部分的工作都是在 predict 步骤中完成的

具体地说：

- 算法的训练不需要花时间，因为其训练过程只是将训练集数据存储起来
- 然而测试要花费大量时间计算，因为每个测试图像需要和所有存储的训练图像进行比较，这显然是一个缺点
- 在实际应用中，我们关注测试效率远高于训练效率。因为 training 可以线下完成

Interview question (covered by Jason):

how to implement the kNN search？

Find the top K shortest distance from your returned unsorted distance array.

方法一：Max-heap

- Time is $O(k + (n-k)logk)$
- Space is $O(k)$

方法二：Min-heap

- Time is $O(n + klog(n))$
- Space is $O(n)$

Further question:

How to implement the kNN search in Spark?

Overall

1. 计算复杂度是多少？

   $n * n * m$

2. 存储复杂度是多少？

   $n * n/2$

3. 怎么去利用这个 matrix？

   a. 现在要给 B 推荐 2 items？

   b. $n(logk) + km + mlog(\hat{k})$

   c. Where k is the k closest friend, and $\hat{k}$ is the number of items to be recommend to users, n is the number of user, m is the number of items

---

复杂度总结

- 离线计算（构建相似 matrix）：

  n * n * m

- 在线计算（用于打分）：

  $n(logk) + km + mlog(\hat{k})$

推荐系统工程结构

```
推荐系统细分四阶段：
召回 + 粗排 + 精排 + ReRanker
```